기초 파이썬

2026. 9. 16일

담당교수: 박진호

서강대학교 경제대학원

2026년 1학기

4.2 맷플롯립(Matplotlib)

4.2.1 Matplotlib이란?

□ 데이터 분석에서는 평균, 중앙값, 표준편차와 같은 통계량을 이용하여 데이터의 특징을 요약할 수 있음

□ 그러나 이러한 숫자만으로는 데이터의 분포나 변수 간 관계를 충분히 파악하기 어려운 경우가 있음

□ 예) 두 집단의 평균소득이 같더라도 한 집단은 평균 주변에 소득이 집중되어 있고 다른 집단은 소득격차가 매우 클 수 있음

▸ 또한 두 변수의 평균과 표준편차만으로는 두 변수 사이에 양(+) 또는 음(-)의 관계가 있는지를 쉽게 알기 어려움

□ 따라서 데이터 분석에서는 기술통계량과 함께 그래프를 이용하여 데이터를 시각적으로 확인하는 과정이 중요

□ 파이썬에서는 데이터를 시각화하기 위한 여러 라이브러리가 있으며, 가장 기본적인 것이 Matplotlib

□ Matplotlib은 선 그래프, 산점도, 히스토그램, 박스플롯, 막대그래프 등 다양한 그래프를 작성할 수 있음

□ 이후 살펴볼 Seaborn도 Matplotlib을 기반으로 함

□ Matplotlib은 다음과 같이 불러옴

```
import matplotlib.pyplot as plt
```

- matplotlib.pyplot에는 그래프를 작성하는 여러 함수가 포함되어 있으며, 일반적으로 'plt'라는 별칭을 사용

4.2.2 Matplotlib의 기본적인 시각화 과정

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(5)
y = [3, 6, 9, 12, 15]

plt.plot(x, y)
plt.savefig("plot.png")
plt.show()
```

□ 먼저 아래 코드는 다음과 같은 값을 만듦

```
x = np.arange(5)
```

```
[0 1 2 3 4]
```

□ 따라서 'x'와 'y'는 각각 다음 값을 가짐

```
x = [0, 1, 2, 3, 4]
y = [3, 6, 9, 12, 15]
```

□ plt.plot(x, y)는 x를 가로축, y를 세로축으로 하여 두 변수의 관계를 선 그래프로 나타냄

▸ 즉, 다음의 좌표가 그래프에 표시

x	y
0	3
1	6
2	9
3	12
4	15

▸ 이 점들을 선으로 연결한 것이 plot()의 기본적인 결과

□ plt.savefig("plot.png")는 작성한 그래프를 파일로 저장하고, plt.show()는 그래프를 화면에 출력

□ 따라서 Matplotlib의 기본적인 작업 과정은 다음과 같이 이해할 수 있음

① 데이터 준비 → ② 그래프 종류 선택 → ③ 그래프 작성 → ④ 필요한 요소 추가 → ⑤ 저장 또는 출력

<참고> Jupyter Notebook에서 plt.show()는 반드시 필요한가?

□ Jupyter Notebook이나 Colab에서는 코드 셀의 실행이 끝나면 그래프가 자동으로 출력되는 경우가 많기 때문에 plt.show()를 생략해도 그림이 나타날 수 있다.

□ 그러나 일반적인 Python 프로그램에서는 자동으로 출력되지 않을 수 있으며, 무엇보다 코드에서 "여기까지가 하나의 그래프이다" 라는 사실을 명확하게 나타낼 수 있다.

□ 따라서 이 교재에서는 다음과 같이 plt.show()를 사용하는 것을 기본으로 한다.

```
plt.plot(x, y)
plt.show()
```

4.2.3 그래프의 기본 구성요소

□ 그래프가 어떤 자료를 나타내는지 알 수 있도록 다음과 같이 제목(title), x축 이름(xlabel), y축 이름(ylabel) 등을 추가

```
x = np.arange(5)
y = [3, 6, 9, 12, 15]

plt.plot(x, y)

plt.title("Sample Plot")
plt.xlabel("X")
plt.ylabel("Y")

plt.show()
```

□ 여러 개의 선을 동시에 표시할 경우에는 label과 legend()를 이용하여 각 선이 무엇을 의미하는지 표시할 수 있음

```
x = np.arange(5)

y1 = [3, 6, 9, 12, 15]
y2 = [5, 4, 3, 2, 1]

plt.plot(x, y1, label="Data 1")
plt.plot(x, y2, label="Data 2")

plt.xlabel("X")
plt.ylabel("Y")
plt.legend()

plt.show()
```

□ label= 은 각 선에 이름을 부여하고, plt.legend()는 이 이름을 이용하여 범례(legend)를 표시

□ 따라서 그래프의 기본적인 구성요소는 다음과 같다.

코드	기능
plt.title()	그래프 제목
plt.xlabel()	x축 이름
plt.ylabel()	y축 이름
label=	그래프의 이름 지정
plt.legend()	범례 표시
plt.show()	그래프 출력
plt.savefig()	그래프 저장

4.2.4 그래프의 선과 점 조정

□ 그래프의 선 모양, 점 모양 등을 변경하면 여러 자료를 보다 더 쉽게 구별할 수 있음

□ 예) 아래와 같이 작성 가능

```
plt.plot( x, y, linestyle="--", marker="o" )
plt.show()
```

- linestyle="--" 는 선을 점선 형태로 표시, marker="o"는 각 관측치를 원형 점으로 표시
- 선의 굵기도 조정

```
plt.plot( x, y, linewidth=2 )
```

<참고> 색상과 선 종류

□ Matplotlib에서는 color, linestyle, marker 등의 옵션을 이용하여 그래프 모양을 다양하게 변경할 수 있음

```
plt.plot( x, y, color="red", linestyle="--", marker="o" )
```

4.2.5 Figure와 Axes

□ Matplotlib의 그래프는 크게 Figure와 Axes로 구성됨

- Figure = 그림 전체가 놓이는 공간,
  Axes = 실제 그래프가 그려지는 영역이라고 생각할 수 있다.

□ 예) fig, ax = plt.subplots() 를 실행하면 하나의 Figure와 하나의 Axes가 만들어짐

Figure
└── Axes
    ├── x축
    ├── y축
    └── 실제 그래프

□ 이제 plt.plot() 대신 ax.plot()을 사용할 수 있음

```
fig, ax = plt.subplots()

ax.plot(x, y)
ax.set_title("Sample Plot")
ax.set_xlabel("X")
ax.set_ylabel("Y")

plt.show()
```

□ 앞에서 배운 plt.plot(x, y)와 목적은 비슷하지만, ax.plot() 방식은 어느 그래프에 그림을 그리는지를 명확하게 지정할 수 있다는 장점이 있음

▸ 이는 여러 개의 그래프를 동시에 작성할 때 특히 유용

<참고> plt.plot()과 ax.plot() 중 어느 것을 사용해야 하는가?

□ 처음 그래프를 배울 때는 아래 방식이 간단

```
plt.plot()
plt.xlabel()
plt.ylabel()
```

□ 그러나 여러 그래프를 함께 배치하거나 그래프를 세밀하게 조정하려면 아래 방식이 편리

```
fig, ax = plt.subplots()
ax.plot()
```

□ 따라서 기초에서는 plt 방식으로 시작하고, 여러 그래프를 다룰 때 fig, ax 방식으로 확장한다고 이해

4.2.6 여러 그래프를 하나의 Figure에 나타내기

□ 서로 다른 두 그래프를 나란히 비교하고 싶을 수 있다. 이때 subplots()를 이용

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
```

▸ 여기서 1=행의 수, 2=열의 수 이므로 하나의 Figure 안에 그래프 두 개가 좌우로 배치됨

□ 예) 아래와 같이 작성

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].plot(x, y1)
ax[0].set_title("Data 1")

ax[1].plot(x, y2)
ax[1].set_title("Data 2")

plt.show()
```

4.2.7 산점도(Scatter Plot)

□ 산점도는 두 변수 사이의 관계를 시각적으로 살펴보기 위한 그래프

▸ 각 관측치는 하나의 점으로 표시

□ 예) 광고비와 매출액 자료가 있다고 하자.

```
advertising = [10, 20, 30, 40, 50]
sales = [15, 25, 28, 40, 48]

plt.scatter(advertising, sales)

plt.xlabel("Advertising")
plt.ylabel("Sales")
plt.show()
```

▸ 각 점은 하나의 관측치를 뜻함.
(10, 15)라는 점은 광고비가 10일 때 매출액이 15였다는 의미

□ 산점도에서 점들이 대체로 오른쪽 위 방향으로 나타난다면, 두 변수 사이에 양(+)의 관계가 있을 가능성

- 반대로 오른쪽 아래 방향으로 나타난다면 음(-)의 관계가 있을 가능성

□ 주의

- 산점도에서 두 변수 사이에 관계가 나타난다고 해서 한 변수가 다른 변수의 원인이라고 결론 내릴 수는 없다.
- 산점도는 변수 간 관계 또는 상관관계를 탐색하는 도구이지, 인과관계를 확인하는 도구는 아니다.

◆ 산점도에서 점이 겹치는 경우

□ 관측치가 많으면 점들이 서로 겹쳐 데이터가 얼마나 밀집되어 있는지 파악하기 어려움

□ 이 경우 alpha를 이용하여 점의 투명도를 조절

```
plt.scatter( advertising, sales, alpha=0.5 )
```

□ alpha는 일반적으로 0과 1 사이의 값을 사용. 값이 작을수록 점이 더 투명하게 표시됨

4.2.8 히스토그램(Histogram)

□ 산점도가 두 변수의 관계를 나타낸다면, 히스토그램은 하나의 변수의 분포를 살펴보기 위한 대표적인 그래프

□ 히스토그램에서는 전체 자료의 범위를 여러 구간으로 나눈 뒤, 각 구간에 얼마나 많은 관측치가 포함되는지를 막대로 나타냄

□ 예) 다음과 같은 자료가 있다고 하자.

```
x = [11, 11, 12, 13, 22,
     26, 33, 43, 43, 48]
```

□ 히스토그램을 그리면:

```
plt.hist(x)

plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

(1) Bin이란?

□ 히스토그램에서 가장 중요한 개념 가운데 하나가 bin

□ Bin은 자료를 구분하기 위해 설정한 구간(interval)

□ 예) 자료를 다음 네 구간으로 나눈다고 하자.

10~20
20~30
30~40
40~50

□ 그러면 각 구간에 들어가는 관측치의 개수를 계산하여 막대의 높이로 나타냄

□ bins를 직접 지정할 수도 있음

```
plt.hist(x,  bins=4)
plt.show()
```

▸ bins=4는 자료의 범위를 4개의 구간으로 나누어 히스토그램을 작성하라는 의미

(2) Bin의 수에 따라 그래프가 달라짐

□ 같은 데이터라도 bin의 개수를 어떻게 정하느냐에 따라 히스토그램의 모양이 달라짐

- 아래 두 코드는 같은 자료를 이용하지만 서로 다른 모양의 히스토그램을 만들 수 있음

```
plt.hist(x, bins=3)
plt.show()
```
와
```
plt.hist(x, bins=8)
plt.show()
```

- Bin이 너무 적으면 분포의 세부적인 특징이 사라질 수 있고, 너무 많으면 자료가 지나치게 잘게 나뉘어 전체적인 분포를 파악하기 어려울 수 있음
- 따라서 히스토그램을 해석할 때에는 bin의 설정도 함께 고려해야 함

4.2.9 빈도 히스토그램과 밀도 히스토그램

□ 관측치 수가 다른 두 집단의 분포를 비교할 때 단순 빈도 히스토그램은 오해를 줄 수 있음

(1) 빈도 히스토그램

□ 일반적인 히스토그램에서는 각 막대의 높이가 해당 구간에 들어 있는 관측치의 수(frequency)를 표시

□ 예) 전체 관측치가 10개이고 어떤 구간에 3개의 관측치가 있다면, 그 구간의 빈도는 3

- 그러나 A집단의 관측치가 1,000개이고 B집단의 관측치가 100개라면, 단순한 빈도만으로 두 집단의 분포를 비교하기 어려움

- 표본이 많은 집단의 막대가 단순히 관측치 수 때문에 더 높아질 수 있기 때문

(2) 밀도 히스토그램

□ 이 경우 밀도 히스토그램(density histogram)을 이용할 수 있음

```
plt.hist(x, bins=4, density=True)
plt.show()
```

□ 밀도 히스토그램에서는 y축이 단순한 관측치 수가 아니라 밀도를 표시

□ 어떤 bin의 관측치 비율이 $\frac{\text{해당 } bin\text{의 관측치 수}}{N}$ 이고 bin의 너비가 w라면 밀도는

$$Density = \frac{\text{해당 } bin\text{의 관측치 비율}}{bin\text{의 너비}} = \frac{\text{해당 } bin\text{의 관측치 비율}}{w}$$

□ 따라서

$$Density = \frac{\text{해당 } bin\text{의 관측치 수}}{N \times bin\text{의 너비}}$$ 가 된다.

(3) 왜 밀도를 이렇게 정의하는가?

□ 막대의 면적은 면적 = 높이 × 너비

□ 밀도 히스토그램에서 높이는 밀도이므로 면적 = Density × w

□ 앞의 밀도 정의를 대입하면

$$\text{면적} = \frac{\text{해당 } bin\text{의 관측치 비율}}{w} \times w \text{ 이므로}$$

"각 bin의 면적 = 해당 bin에 속하는 관측치의 비율" 이 된다.

□ 따라서 모든 bin의 면적을 합하면 1 이 된다.

<참고> 빈도와 밀도를 혼동하지 않기

□ 밀도 히스토그램에서 막대의 높이 자체가 확률이나 비율은 아님

□ 비율을 나타내는 것은 높이 × 너비, 즉 막대의 면적

□ 따라서 bin의 너비가 작으면, 밀도값이 1보다 커질 수도 있음

4.2.10 박스플롯(Box Plot)

□ 히스토그램이 분포의 모양을 비교적 자세하게 보여준다면, 박스플롯은 분포의 주요 특징을 몇 개의 통계량으로 간결하게 나타냄

□ 박스플롯을 이해하기 위해서는 먼저 사분위수를 알아야 함

□ 자료를 작은 값부터 큰 값까지 정렬했을 때, 아래와 같다.

- Q_1 = 제1사분위
- Q_2 = 중앙값(Median)
- Q_3 = 제3사분위수

- 사분위범위(interquartile range)는 $IQR = Q_3 - Q_1$로 정의

□ 박스는 Q_1에서 Q_3까지의 범위를 나타내므로 전체 관측치 가운데 50%가 위치하는 범위를 보여줌

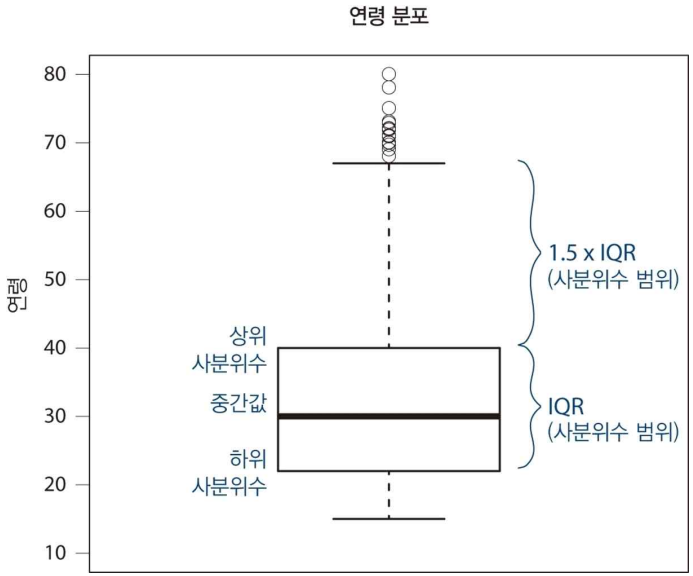
◆ **이상치의 판단:** 다음의 경계를 이용하여 잠재적인 이상치를 판단

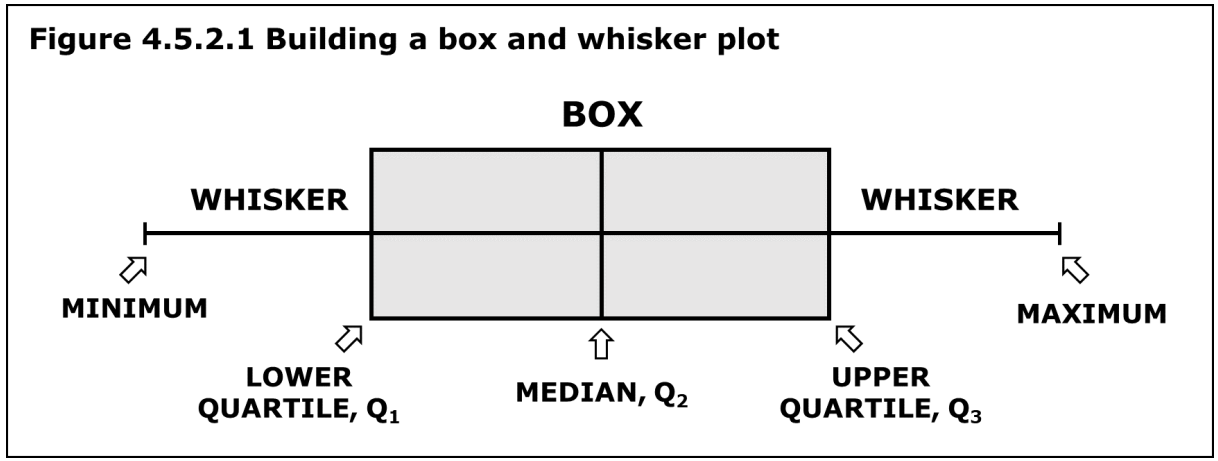
- 하한: $Q_1 - 1.5IQR$
- 상한: $Q_3 + 1.5IQR$

□ 이 범위를 벗어난 값은 박스플롯에서 별도의 점으로 표시될 수 있음

- 박스플롯을 통해 중앙값, 자료의 퍼짐, 비대칭성, 잠재적인 이상치 등을 동시에 확인 가능

```
plt.boxplot(x)
plt.show()
```





<참고> 수염(whisker)은 항상 최소값과 최대값인가?

□ 아니다

□ Matplotlib의 일반적인 박스플롯에서 수염은 단순히 데이터의 최소값과 최대값까지 무조건 연결되는 것이 아님

□ 일반적으로 $Q_1 - 1.5IQR$과 $Q_3 + 1.5IQR$ 의 범위 안에 존재하는 가장 바깥쪽 관측치까지 수염이 연장됨

□ 그 범위를 벗어나는 관측치는 별도의 점으로 표시됨

4.2.11 히스토그램과 박스플롯의 비교

그래프	주요 목적	장점
히스토그램	분포의 전체적인 모양 확인	집중, 비대칭, 여러 봉우리 등을 확인하기 쉬움
박스플롯	분포를 요약하여 비교	중앙값, IQR, 이상치를 간결하게 확인
산점도	두 변수의 관계 확인	변수 간 양·음의 관계와 특이 관측치 확인

4.2.12 막대그래프(Bar Plot)

□ 히스토그램과 막대그래프는 모두 막대를 사용하기 때문에 비슷해 보이지만 목적이 다름

□ 예) 국가별 수출액이 다음과 같다고 하자.

```
country = [“Korea“, “US“, “Japan“]
export = [100, 150, 120]

plt.bar(country, export)

plt.xlabel(“Country“)
plt.ylabel(“Export“)
plt.show()
```

□ 막대그래프는 국가처럼 서로 구분되는 범주(category)의 값을 비교

□ 반면 히스토그램은 연속형 자료를 여러 구간으로 나누어 분포를 나타냄

4.2.13 선 그래프(Line Plot)

□ 선 그래프는 시간에 따라 값이 어떻게 변하는지를 나타낼 때 많이 사용

□ GDP, 물가상승률, 실업률, 금리, 주가 등 대부분의 시계열 자료를 선 그래프로 먼저 확인

```
year = [2020, 2021, 2022, 2023, 2024]
GDP_growth = [2.3, 2.5, 3.0, 2.4, 1.9]

plt.plot(year, GDP_growth, marker="o")

plt.xlabel("Year")
plt.ylabel("GDP Growth (%)")
plt.show()
```

□ 선 그래프에서는 개별 값뿐 아니라 시간에 따른 변화의 방향과 추세(trend)를 확인

□ 따라서 다음과 같이 그래프의 목적을 구분할 수 있음

- 시간에 따른 변화 → 선 그래프
- 두 변수의 관계 → 산점도
- 하나의 연속형 변수의 분포 → 히스토그램
- 분포와 이상치의 요약 → 박스플롯
- 범주별 크기 비교 → 막대그래프

4.2.14 그래프 저장하기

□ 분석 결과를 보고서나 논문에 사용하려면 그래프를 파일로 저장해야 함

```
plt.plot(year, GDP_growth)

plt.xlabel("Year")
plt.ylabel("GDP Growth (%)")

plt.savefig("GDP_growth.png")
plt.show()
```

□ 해상도를 지정할 수도 있음

```
plt.savefig( "GDP_growth.png", dpi=300, bbox_inches="tight" )
```

- 여기서 dpi는 그림의 해상도를 지정하고, bbox_inches="tight"는 그래프 주변의 불필요한 여백을 줄이는 데 사용한다.

4.2.16 실제 데이터를 이용한 분포 비교

◆ Brexit 관련 자료를 이용하여 탈퇴(leave) 집단과 잔류(stay) 집단의 연령분포를 비교해 보자.

(1) 데이터 불러오기

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("../Data/데이터파일명.csv")
```

(2) 데이터 구조 확인

```
df.head()
```

- leave = 1 : 탈퇴(leave)
- leave = 0 : 잔류(stay)
- leave = NaN : don't know 등

	vote	leave	education	age
0	leave	1.0	3.0	60
1	leave	1.0	NaN	56
2	stay	0.0	5.0	73
3	leave	1.0	4.0	64
4	don’t know	NaN	2.0	68

df.info()

(3) 분석할 변수의 기술통계량 확인

df[“age“].describe()

□ describe()를 이용하면 평균, 표준편차, 최소값, 사분위수, 최대값 등을 확인할 수 있음

□ 그러나 기술통계량만으로는 분포의 모양을 충분히 알기 어렵다. 따라서 히스토그램을 함께 확인

(4) 두 집단으로 데이터를 구분

□ 변수가 0과 1로 구분되어 있다면 Boolean indexing을 이용할 수 있음

```
group1 = df[df[“leave“] == 1]
group2 = df[df[“leave“] == 0]
```

- df[“leave“] == 1 은 각 관측치가 조건을 만족하는 행만 선택
- 따라서 group1과 group2에는 두 집단의 관측치가 각각 저장

4.2.17 두 집단의 히스토그램 비교

□ 두 집단의 히스토그램을 각각 나타내 비교하는 것은 불편하므로 두 그래프를 하나의 Figure에 나란히 배치

- 두 집단을 비교하려면 앞에서 만들었던 subplots()을 이용
- 비교 대상은 age 변수의 분포

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].hist(group1["age"], bins=10)
ax[0].set_title("Leave")
ax[0].set_xlabel("Age")
ax[0].set_ylabel("Frequency")

ax[1].hist(group2["age"], bins=10)
ax[1].set_title("Stay")
ax[1].set_xlabel("Age")
ax[1].set_ylabel("Frequency")

plt.show()
```

- plt.subplots(1, 2)에서 1은 1행, 2는 2열을 의미
- ax[0]은 왼쪽 그래프, ax[1]은 오른쪽 그래프를 의미
- leave는 집단을 나누는 변수이고, age는 분포를 비교할 변수

4.2.18 표본의 크기가 다르면 어떻게 비교할까?

□ 두 집단의 관측치 수가 크게 다르다고 하자.

□ 예) Group 1 : 1,000개, Group 2 : 200개 라면,
단순한 빈도 히스토그램에서는 Group 1의 막대가 전반적으로 더 높게 나타날 수 있음

□ 그러나 이것이 반드시 Group 1의 데이터가 특정 구간에 상대적으로 더 집중되어 있다는 의미는 아님. 단순히 전체 관측치가 더 많기 때문일 수도 있음

□ 따라서 서로 다른 크기의 두 표본에서 분포의 모양 자체를 비교하려면, 밀도 히스토그램을 이용할 수 있음

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].hist( group1[“age“], density=True )
ax[1].hist( group2[“age“], density=True )

plt.show()
```

4.2.19 두 집단을 하나의 그래프에서 비교하기

□ 두 집단의 분포를 하나의 그래프에 겹쳐서 나타낼 수도 있음

```
plt.hist( group1[“variable“], bins=10, density=True, alpha=0.5, label=“Group 1“ )
plt.hist( group2[“variable“], bins=10, density=True, alpha=0.5, label=“Group 2“ )

plt.legend()
plt.show()
```

코드	의미
bins=10	자료를 10개의 구간으로 구분
density=True	빈도 대신 밀도로 표현
alpha=0.5	막대를 반투명하게 표시
label=	각 집단의 이름 지정
legend()	범례 표시

▸ alpha를 사용하는 이유: 두 히스토그램을 겹쳐 그리면 막대가 서로 가릴 수 있기 때문에 투명도를 높여 두 분포를 동시에 볼 수 있도록 하기 위함

<참고> 같은 x축과 y축을 사용하는 이유

□ 두 집단의 그래프를 나란히 비교할 때 각각의 축 범위가 다르면 분포의 차이를 정확하게 비교하기 어려울 수 있음

□ Brexit 자료에서 탈퇴(leave) 및 잔류(stay) 집단의 연령분포를 비교하기 위해 두 집단을 구분

```
group1 = df[df[“leave“] == 1]
group2 = df[df[“leave“] == 0]
```

□ 두 집단의 연령분포를 하나의 Figure에 나란히 표시

```
fig, ax = plt.subplots( 1, 2, figsize=(10, 4), sharex=True, sharey=True )

ax[0].hist(group1[“age“], bins=10)
ax[0].set_title(“Leave“)
ax[0].set_xlabel(“Age“)
ax[0].set_ylabel(“Frequency“)

ax[1].hist(group2[“age“], bins=10)
ax[1].set_title(“Stay“)
ax[1].set_xlabel(“Age“)
plt.show()
```

□ sharex=True와 sharey=True는 각각 동일한 x축 및 y축의 범위와 눈금을 사용하도록 함

□ 그런데, leave와 stay의 표본 수가 크게 다르면 빈도가 달라져 막대 높이의 차이가 분포 차이인지 단순한 표본 수 차이인지 구분하기 어려움

□ 분포의 형태 자체를 비교하려는 목적이라면 앞에서 배운 density=True와 연결

```
ax[0].hist( group1[“age“], bins=10, density=True )
ax[0].set_title(“Leave“)
ax[0].set_xlabel(“Age“)
ax[0].set_ylabel(“Density“)

ax[1].hist( group2[“age“], bins=10, density=True )
ax[1].set_title(“Stay“)
ax[1].set_xlabel(“Age“)
```

4.3 시본(Seaborn)을 이용한 데이터 시각화

4.3.1 Seaborn이란?

□ Matplotlib은 다양한 그래프를 자유롭게 만들 수 있다는 장점이 있지만, 그래프의 모양을 세밀하게 설정하려면 코드가 길어질 수 있음

- 특히 Pandas의 DataFrame을 이용하여 여러 집단을 비교하거나 통계적인 관계를 시각화할 때에는 상당한 코드를 작성해야 하는 경우가 있음

□ Seaborn은 이러한 작업을 보다 편리하게 수행하기 위한 데이터 시각화 라이브러리

□ Seaborn은 Matplotlib을 기반으로 작동하면서 Pandas의 DataFrame과 직접 연결하여 사용할 수 있음

```
import seaborn as sns
import matplotlib.pyplot as plt
```

- 관례적으로 Seaborn은 'sns'라는 이름으로 불러옴

◆ Matplotlib과 Seaborn의 관계

□ Matplotlib은 그래프를 만드는 기본 도구이고, Seaborn은 그 위에서 통계 데이터를 보다 편리하게 시각화하는 도구

□ 실제로 Seaborn으로 그래프를 만든 뒤에도 Matplotlib의 함수를 이용하여 제목이나 축 이름을 변경할 수 있음

□ 따라서 실제 분석에서는 두 라이브러리를 함께 사용하는 경우가 많음

4.3.2 Seaborn의 기본적인 문법

□ Seaborn을 처음 사용할 때 가장 중요한 특징은 DataFrame의 열 이름을 직접 지정하여 그래프를 그릴 수 있다는 것

□ 예) 다음 데이터를 사용하여 두 변수의 관계를 나타내기 위해 코드를 작성

Year	GDP Growth	Unemployment rate
2020	-2.2	8.1
2021	5.8	5.4
2022	1.9	3.6
2023	2.9	3.6

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터 생성
year = [2020, 2021, 2022, 2023]
GDP_growth = [2.3, 2.5, 3.0, 2.4]
Unemployment_rate = [8.1, 5.4, 3.6, 3.6]

# DataFrame 생성
df = pd.DataFrame({
    "Year": year,
    "GDP_Growth": GDP_growth,
    "Unemployment_rate": Unemployment_rate })
df

sns.scatterplot( data=df, x="GDP_Growth", y="Unemployment_rate" )

plt.title("GDP Growth and Unemployment")
plt.xlabel("GDP Growth (%)")
plt.ylabel("Unemployment Rate (%)")
plt.show()
```

4.3.3 Seaborn에서는 어떤 그래프를 사용하는가?

분석 목적	대표적인 함수	예
두 변수의 관계	scatterplot(), lineplot()	GDP 성장률과 실업률
범주별 비교	barplot(), countplot()	국가별 성장률
분포 확인	histplot(), boxplot()	소득분포
회귀관계 확인	regplot()	GDP 성장률과 실업률
여러 변수의 관계	heatmap() 등	변수 간 상관관계

4.3.4 관계 그래프(Relational Plot)

□ 관계 그래프는 두 변수 사이의 관계를 살펴볼 때 사용

- 대표적 예) 산점도(scatter plot), 선 그래프(line plot)

□ 두 그래프는 비슷해 보일 수 있지만 목적은 다름

- 산점도 : 두 변수 사이의 관계
- 선 그래프 : 시간이나 순서에 따른 변화

4.3.4.1 scatterplot(): 두 변수의 관계

□ 미국 GDP 성장률과 실업률 자료를 이용하여 두 변수의 관계를 산점도로 분석

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("US_GDP_Unemp.csv")
sns.scatterplot( data=df, x="GDP Growth", y="Unemployment rate" )

plt.xlabel("GDP Growth (%)")
plt.ylabel("Unemployment Rate (%)")
plt.title("GDP Growth and Unemployment")
```

□ 어떤 점의 좌표가 (3.0, 4.5)라면 그 해의 GDP 성장률이 3%, 실업률이 4.5%라는 뜻

◆ <참고> 관측치에 연도 표시하기

```
for i in range(len(df)):
    plt.text( df["GDP Growth"][i] + 0.1, df["Unemployment rate"][i] + 0.1, str(df["Year"][i]) )
                # x 좌표                    # y 좌표                              # 표시할 문자

plt.show()
```

① len(df): 관측치의 개수

- len(df)는 DataFrame df의 행의 수(전체 관측치의 수)를 반환 → 4

② range(len(df)): 관측치 번호 만들기 → len(df)=4라면 range(4)가 됨

③ for i in ...: 모든 관측치를 하나씩 반복 → 각 연도의 관측치를 하나씩 처리

- i의 값이 다음과 같이 차례로 변하면서 아래의 코드를 반복 실행

```
i = 0
i = 1
i = 2
i = 3
```

④ plt.text(): 그래프에 문자 표시

- plt.text()는 그래프의 특정 위치에 문자(text)를 표시하는 함수
- 세 가지 정보가 필요: plt.text(x좌표, y좌표, 표시할 문자)

⑤ df["GDP Growth"][i] + 0.1

- i번째 관측치의 GDP 성장률을 가져옴
- GDP 성장률은 산점도의 x축 값이므로 이를 연도를 표시할 x좌표로 사용
- + 0.1은 연도 글자가 점과 겹치지 않도록 점의 오른쪽으로 조금 이동시킴

⑥ df["Unemployment rate"][i] + 0.1

- i번째 관측치의 실업률을 가져옴
- 실업률은 산점도의 y축 값이므로 이를 연도를 표시할 y좌표로 사용
- 여기에서도 + 0.1을 하여 연도 글자를 점보다 약간 위쪽에 표시

⑦ str(df["Year"][i]): 연도를 문자로 변환

- df["Year"][i]는 i번째 관측치의 연도를 가져옴
- plt.text()에 표시할 글자로 사용하기 위해 str()을 이용하여 연도를 문자열(string)로 변환

4.3.4.2 lineplot(): 시간에 따른 변화

□ 산점도와 달리 선 그래프는 시간에 따른 변수의 변화를 살펴보는 데 적합

□ 같은 미국 자료에서 GDP 성장률의 시간적 변화를 살펴보자.

```
sns.lineplot( data=df, x="Year", y="GDP Growth", marker="o" )

plt.xlabel("Year")
plt.ylabel("GDP Growth (%)")
plt.title("US GDP Growth")

plt.show()
```

- sns.scatterplot(data=df, x="GDP Growth", y="Unemployment rate")
  은 GDP 성장률과 실업률 사이에 어떤 관계가 있는가?를 묻는 그래프

- sns.lineplot(data=df, x="Year", y="GDP Growth")
  은 GDP 성장률은 시간에 따라 어떻게 변화했는가?를 묻는 그래프

4.3.4.3 하나의 그래프에 두 변수 나타내기

□ GDP 성장률과 실업률의 시간적 움직임을 함께 비교하고 싶다면, 두 개의 lineplot()을 사용

```
plt.figure(figsize=(10, 6))

sns.lineplot( data=df, x="Year", y="GDP Growth", label="GDP Growth", marker="o" )
sns.lineplot( data=df, x="Year", y="Unemployment rate", label="Unemployment Rate", marker="o" )

plt.xlabel("Year")
plt.ylabel("Percent")
plt.title("Economic Trends")
plt.legend()

plt.show()
```

□ 여기서는 Seaborn과 Matplotlib을 함께 사용하고 있다는 점도 중요

4.3.5 relplot()은 무엇인가?

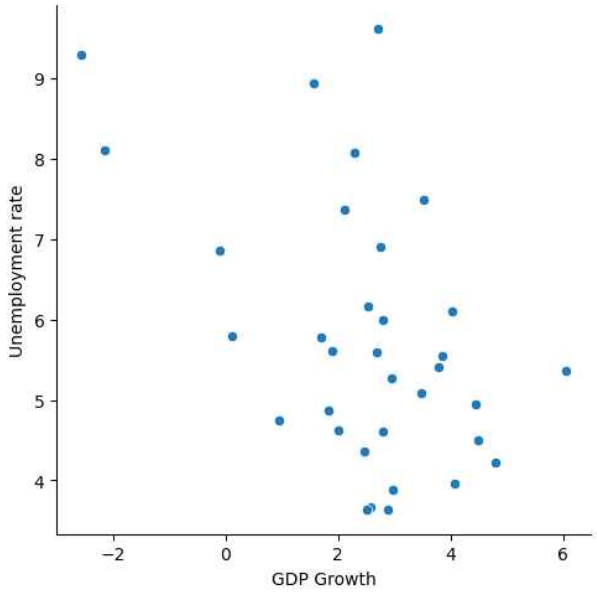
□ relplot()은 관계 그래프를 만드는 더 상위 수준의 함수

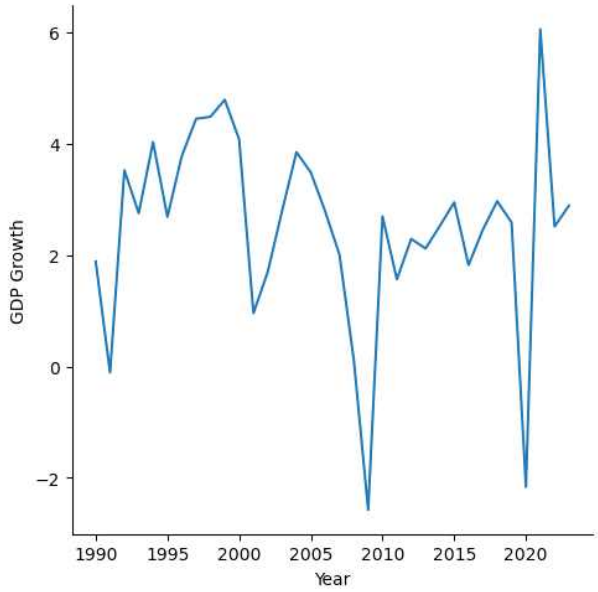
□ 예) 산점도를 만듦

sns.relplot(data=df, x="GDP Growth", y="Unemployment rate", kind="scatter")

□ 예) kind를 바꾸면 선 그래프를 만들 수도 있음

sns.relplot(data=df, x="Year", y="GDP Growth", kind="line")





Q: “이미 scatterplot()과 lineplot()이 있는데 왜 relplot()이 또 필요한가?”

- relplot()의 장점은 집단별로 여러 그래프를 쉽게 나누어 그릴 수 있다는 것
- 예) 어떤 데이터에 ‘country‘ 변수가 있다면, 아래와 같이 국가별 그래프를 자동으로 나누어 만들 수 있음

```
sns.relplot( data=df, x=“Year“, y=“GDP Growth“, col=“country“, kind=“line“ )
```

<참고> Axes-level과 Figure-level 함수

□ Seaborn 함수는 크게 두 유형으로 나눌 수 있음

(1) Axes-level function

sns.scatterplot()
sns.lineplot()
sns.boxplot()
sns.barplot()

□ 하나의 'Axes'에 그래프를 그리는 함수. 따라서 Matplotlib의 subplot과 쉽게 결합할 수 있음

```
fig, ax = plt.subplots(1, 2)
sns.scatterplot( data=df, x="GDP Growth", y="Unemployment rate", ax=ax[0] )
sns.lineplot( data=df, x="Year", y="GDP Growth", ax=ax[1] )
plt.show()
```

□ 여기서 ax=ax[0]는 "첫 번째 그래프 영역에 이 산점도를 그려라." 라는 의미

(2) Figure-level function

□ 대표적인 예)

sns.relplot()
sns.catplot()
sns.displot()

□ 이 함수들은 Figure와 여러 Axes를 내부적으로 만들어 관리

□ 특히 col, row 등을 이용하여 여러 집단의 그래프를 자동으로 배치할 때 편리하다.

□ 따라서 하나의 그래프를 직접 제어 → axes-level

여러 그래프를 집단별로 자동 배치 → figure-leve

<참고> Seaborn 그래프의 테두리

□ Matplotlib의 그래프 영역(Axes)에는 기본적으로 네 방향의 테두리가 있으며, 이 테두리를 spine(스파인)이라고 함

□ Seaborn은 통계 그래프를 좀 더 간결하게 보여주기 위해 기본 스타일에서 위쪽과 오른쪽 spine을 표시하지 않는 경우가 있음

▸ 이는 데이터나 분석 결과의 차이가 아니라 그래프의 시각적 표현 방식(style)의 차이

□ 네 방향 테두리를 모두 표시하려면, relplot()이 반환하는 객체를 받아서 다음과 같이 코딩

```
g = sns.relplot( data=df, x="GDP Growth", y="Unemployment rate" )

for ax in g.axes.flat:
    ax.spines["top"].set_visible(True)
    ax.spines["right"].set_visible(True)
```

4.3.6 hue: 집단을 색으로 구분하기

◇ GDP 성장률과 실업률의 관계는 글로벌 금융위기 이전과 이후에도 비슷하게 나타나는가

◇ 실업률이 올라가면 경제성장률은 떨어진다는 GDP 성장률과 실업률 사이의 음(-)의 관계를 설명하는가? → 오쿤의 법칙?

□ Seaborn에서는 hue를 이용하여 관측치를 집단별로 서로 다른 색으로 구분

□ 앞서 사용한 미국의 GDP 성장률과 실업률 자료를 GFC 이전과 이후의 두 시기로 구분

❶ Year 변수(컬럼)를 기준으로 데이터가 Year ≤ 2007이면 Before GFC, 그렇지 않으면 After GFC를 새로운 변수 Period에 저장

```
import numpy as np

df[“Period“] = np.where( df[“Year“] <= 2007, “Before 2008“, “2008 and After“ )
```

- np.where() 함수는 조건에 따라 두 개의 값 중 하나를 선택하여 새로운 값을 만드는 조건문(If-Else) 기반의 배열/시리즈 생성 기능을 수행
- np.where(조건문, 조건이 True일 때 값, 조건이 False일 때 값)

❷ 데이터를 확인

```
df[["Year", "GDP Growth", "Unemployment rate", "Period"]].head()
```

❸ hue를 이용하여 두 시기 구분

```
sns.scatterplot( data=df, x="GDP Growth", y="Unemployment rate", hue="Period" )

plt.xlabel("GDP Growth (%)")
plt.ylabel("Unemployment Rate (%)")
plt.title("GDP Growth and Unemployment")

plt.show()
```

<참고> 범례 제목 제거

□ Seaborn에서 hue를 사용하면 변수명이 범례의 제목으로 자동 표시됨

□ 범례 제목이 필요하지 않은 경우 plt.legend(title=None)을 사용

4.3.7 범주형 자료의 시각화

□ 지금까지는 주로 연속형 변수 사이의 관계를 살펴봄

□ 그러나 경제·경영 데이터에는 국가, 산업, 성별, 지역, 기업규모와 같은 범주형 변수(categorical variable)도 많이 포함됨

4.3.7.1 barplot()

□ 어느 국가의 GDP 성장률이 더 높은가?

□ 국가별 GDP 성장률 자료를 사용

Country	GDP Growth	Unemployment
USA	2.5	3.6
UK	1.8	4.8
China	6.1	5.1
India	7.3	6.2
Germany	1.2	4.2

```
data = {
    “Country“: [“USA“, “UK“, “China“, “India“, “Germany“],
    “GDP Growth“: [2.5, 1.8, 6.1, 7.3, 1.2],
    “Unemployment“: [3.6, 4.8, 5.1, 6.2, 4.2]
}

df_cat = pd.DataFrame(data)
```

4.3.7.2 countplot()

□ 이번에는 국가를 선진국과 개발도상국으로 구분한다고 하자.

```
df_cat["Economic Status"] = [
    "Advanced", "Advanced", "Developing", "Developing", "Advanced"
]
```

□ 그러면 자료는 다음과 같이 바뀜

Country	GDP Growth	Unemployment	Economic Status
USA	2.5	3.6	Advanced
UK	1.8	4.8	Advanced
China	6.1	5.1	Developing
India	7.3	6.2	Developing
Germany	1.2	4.2	Advanced

□ 각 범주에 몇 개의 관측치가 있는지를 알고 싶다면 countplot()을 사용할 수 있음

```
sns.countplot( data=df_cat, x="Economic Status" )
plt.show()
```

4.3.8 분포 그래프

4.3.8.1 histplot()

```
sns.histplot( data=df, x="GDP Growth", bins=10 )
plt.show()
```

□ 밀도로 나타내려면:

```
sns.histplot( data=df, x="GDP Growth", bins=10, stat="density" )
plt.show()
```

□ Seaborn에서는 KDE를 함께 표시할 수도 있음

```
sns.histplot( data=df, x="GDP Growth", kde=True )
plt.show()
```

<참고> KDE란?

□ KDE(Kernel Density Estimation)는 관측값의 분포를 매끄러운 곡선으로 추정하여 보이는 방법

□ 히스토그램이 자료를 여러 bin으로 나누어 막대로 나타낸다면, KDE는 분포의 전체적인 모양을 곡선으로 보여줌

□ 따라서 kde=True 를 지정하면 히스토그램과 함께 분포의 대략적인 형태를 쉽게 살펴볼 수 있음

□ 다만 KDE는 실제 관측치를 단순히 선으로 연결한 것이 아니라 추정된 밀도곡선이라는 점에 주의

4.3.8.2 boxplot()

□ 박스플롯도 DataFrame의 열 이름을 직접 이용할 수 있음

```
sns.boxplot( data=df, y="GDP Growth" )

plt.ylabel("GDP Growth (%)")
plt.show()
```

□ 범주별 분포를 비교할 때 Seaborn의 장점이 더욱 분명해짐

□ 예) 여러 지역의 소득분포를 비교한다면, 아래와 같이 작성할 수 있음

```
sns.boxplot( data=df, x="region", y="income" )
plt.show()
```

4.3.9 회귀 그래프: regplot()

□ GDP 성장률과 실업률의 산점도에 직선을 추가해 보자.

```
sns.regplot( data=df, x="GDP Growth", y="Unemployment rate" )

plt.xlabel("GDP Growth (%)")
plt.ylabel("Unemployment Rate (%)")
plt.show()
```

□ regplot()은 관측치를 산점도로 표시하고 두 변수 사이의 선형관계를 나타내는 회귀선 (regression line)을 함께 표시

□ 따라서 scatterplot()에서 보았던 관계의 방향을 보다 명확하게 확인할 수 있음

□ 그러나 회귀선이 나타났다고 해서 인과관계가 확인된 것은 아님

□ 이 그래프는 "두 변수 사이에 어떤 선형적인 관계가 나타나는가?" 를 탐색하는 도구

4.3.10 Matplotlib과 Seaborn을 어떻게 선택할 것인가?

□ 두 라이브러리 중 하나만 선택해야 하는 것은 아님

목적	적합한 방법
간단한 기본 그래프	Matplotlib
그래프를 세밀하게 수정	Matplotlib
제목·축·범례 등의 최종 조정	Matplotlib
DataFrame을 바로 시각화	Seaborn
집단별 분포 비교	Seaborn
통계적 관계의 빠른 탐색	Seaborn

4.4 함수와 튜플

4.4.1 함수

4.4.1.1 함수란?

□ 함수(function)는 특정한 작업을 수행하도록 미리 정의해 놓은 코드의 묶음

- 즉, 어떤 값을 입력받아, 정해진 작업을 수행하고, 그 결과를 반환

4.4.1.2 함수를 사용하는 이유

□ 동일한 작업을 여러 번 수행해야 할 때 코드를 매번 다시 작성하는 것은 비효율적

❶ 함수 미사용시:

```
a = 10
b = 20

result = a + b
print(result)
```

❷ 함수 사용시: 코드를 재사용할 수 있음

```
def add(a, b):
    result = a + b
    return result

add(10, 20)          # 결과: 30
add(100, 50)         # 결과: 150
```

4.4.1.3 함수 정의하기: def

□ def 키워드를 사용하여 함수를 정의

```
def 함수이름(입력값):
    실행할 코드
    return 결과
```

□ 예)

```
def add(a, b):
    result = a + b
    return result
```

- def add(a, b): → add라는 이름의 함수를 정의
- result = a + b → 입력된 두 값을 더하여 result에 저장
- return result → 계산된 결과를 반환

4.4.1.4 매개변수와 인수

□ 매개변수(parameter): 함수를 정의할 때 사용하는 변수 (a, b)

□ 인수(argument): 함수를 호출할 때 실제로 전달하는 값 (10, 20)

def add(a, b) ← 매개변수 (parameter)

add(10, 20) ← 인수 (argument)

4.4.1.5 위치 인수와 키워드 인수

□ 함수에 값을 전달할 때는 두 가지 방식을 사용할 수 있음

❶ 위치 인수(positional argument):

- 위치에 따라 순서대로 전달되는 방식

```
add(10, 20)          # 10은 a에, 20은 b에 순서대로 전달
```

❷ 키워드 인수(keyword argument):

- 이름을 지정하여 전달하는 방식
- 값이 무엇을 의미하는지가 명확해져 코드의 가독성이 높아짐

```
add(a=10, b=20)      # 이름을 직접 지정하여 전달 (순서가 바뀌어도 작동)
```

4.4.1.6 함수의 반환값(return value)

□ 함수는 작업을 수행한 후 결과를 반환할 수 있음

□ 예) 다음 함수를 실행하면 30이라는 결과가 반환됨

```
def add(a, b):
    return a + b

add(10, 20)
```

□ 함수의 결과를 변수에 저장하고 출력

```
result = add(10, 20)

print(result)
```

▸ 결과: 30

4.4.2 List와 Tuple

□ Python에서는 여러 개의 값을 하나로 묶어 저장하기 위해 List(리스트)와 Tuple(튜플)을 사용할 수 있음

□ 리스트(List)와 튜플(Tuple)은 모두 여러 값을 순서대로 저장할 수 있지만, 값을 표현하는 방법과 생성 후 값을 변경할 수 있는지에 차이가 있음

구분	List	Tuple
표현	[ ]	( )
여러 값 저장	가능	가능
순서 존재	있음	있음
Index 사용	가능	가능
생성 후 값 변경	가능	불가능

```
# List
a = [10, 20]

# Tuple
b = (10, 20)
```

❶ List와 Tuple의 공통점

□ List와 Tuple은 모두 여러 개의 값을 하나의 변수에 순서대로 저장할 수 있음

```
a = [10, 20, 30]
b = (10, 20, 30)
```

□ 저장된 값에는 index를 이용하여 접근할 수 있음

```
print(a[0])
print(b[0])
```

▸ 결과: 10
10

❷ List와 Tuple의 차이점

□ 가장 중요한 차이: 생성한 후 저장된 값을 변경할 수 있는지 여부

□ List는 생성한 후 개별 값을 변경할 수 있음

▸ 이때문에 List는 여러 값을 저장하고 필요에 따라 값을 추가・삭제・변경하는 경우에 적합

```
a = [10, 20]

a[0] = 30

print(a)
```

▸ 결과: [30, 20]

□ 반면 Tuple은 생성한 후 개별 값을 변경할 수 없음

```
b = (10, 20)

b[0] = 30

print(b)
```

▸ 결과: 오류 발생

▸ 이처럼 생성된 후 값을 변경할 수 없는 특성을 불변성(immutability)이라고 함

❸ Tuple은 왜 사용하는가?

□ Tuple은 서로 관련된 여러 값을 한 묶음으로 표현하고 그 구성을 유지하고자 할 때 유용

▸ 예) 하나의 점을 나타내는 (x, y) 좌표는 두 개의 값 (3, 5)이 하나의 정보를 구성

```
point = (3, 5)
```

□ Python에서는 함수가 여러 개의 결과를 반환할 때 Tuple이 자주 사용됨

```
def calculate(a, b):
    return a + b, a - b

result = calculate(10, 3)

print(result)
```

▸ 결과: (13, 7)

▸ 함수가 계산한 합 13과 차 7이 하나의 Tuple로 묶여 반환됨

❹ Tuple에 저장된 여러 값은 각각의 변수에 나누어 저장할 수 있음

```
a = (10, 20)

x, y = a

print(x)
print(y)
```

- 결과: 10
  20

- x, y = (10, 20)에서 x ← 10, y ← 20에 각각 저장됨

- 이처럼 하나로 묶여 있는 값을 여러 변수에 나눠 저장하는 것을 unpacking(언패킹)이라고 함

❺ 함수의 여러 반환값과 Tuple unpacking

□ 앞의 calculate() 함수에서도 반환된 두 값을 바로 각각의 변수에 저장할 수 있음

```
def calculate(a, b):
    return a + b, a - b

x, y = calculate(10, 3)

print(x)
print(y)
```

- 결과: 13
  7
- 즉, x, y = calculate(10, 3)은 함수가 반환한 (13, 7)을 unpacking하여 x ← 13, y ← 7로 저장

□ 이러한 방식은 앞으로 그래프를 작성할 때 사용하는 다음 코드에서도 나타남

```
fig, ax = plt.subplots()
```

- plt.subplots()가 반환하는 Figure와 Axes를 fig와 ax에 각각 나누어 저장